

Week 4 demo — fallback captures

Bike-sharing class diagram

Captured with Claude Code, Sonnet 5 (low effort), September 2026

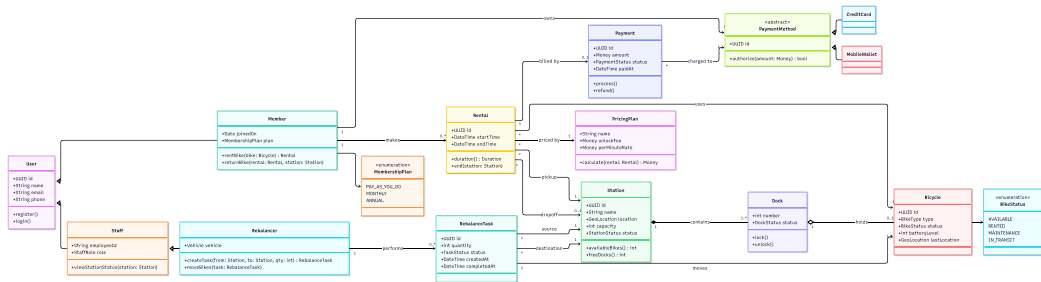
When to use this deck

Switch here if the live AI fails. These are real answers to the demo's own prompts; walk the same defect catalogue against them.

Prompt #1

“Generate a UML class diagram (as Mermaid) for the city bike-sharing app: users rent and return bicycles at stations across a city; payment is by app; staff rebalance bikes between stations.”

AI's answer — the diagram



AI's answer — what it said

Notes

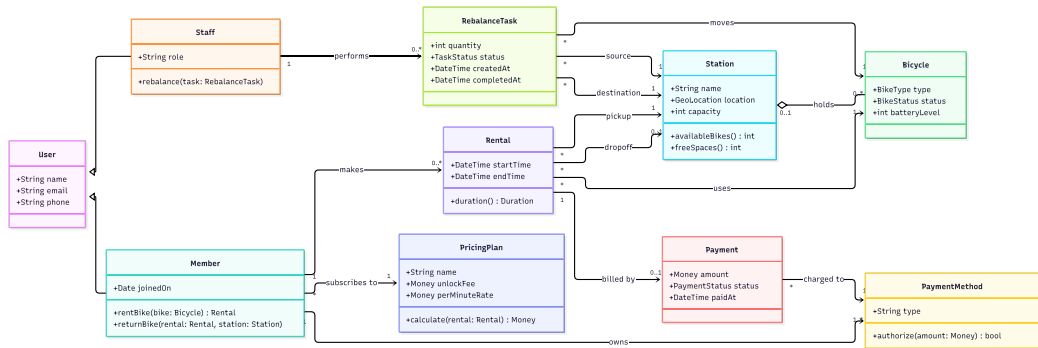
- ▶ **Users:** *Member (riders) and Staff specialize User. Rebalancer is the staff role that moves bikes between stations.*
- ▶ **Rental flow:** *A Rental links a member, a bicycle, a pickup station and a dropoff station. [...]*
- ▶ **Payment:** *Payments happen in the app. Each Payment is charged to a PaymentMethod, either CreditCard or MobileWallet. PricingPlan computes the cost.*
- ▶ **Rebalancing:** *A RebalanceTask moves one or more bicycles from a source station to a destination station.*

I can add other classes if you want them, such as maintenance, reservations or a notification service.

Prompt #2 (same session)

“Revise the class diagram. Drop any class that’s an implementation detail rather than a domain concept — keep only domain classes. Make the Station–Bike relationship an aggregation (a station holds bikes); a bike being ridden is at no station, so fix that multiplicity. Replace every attribute that stores another object’s id with an association.”

AI's answer — the revised diagram



Al's answer — its change list

What changed

▶ **Dropped implementation-detail classes:**

- ▶ *Dock* is gone. A station now holds bikes directly.
- ▶ The *CreditCard* and *MobileWallet* subclasses are gone. *PaymentMethod* keeps a *type* attribute instead.
- ▶ The *Rebalancer* subclass is gone. Rebalancing is now a *Staff* responsibility.
- ▶ The enumeration classes (*MembershipPlan*, *BikeStatus*) are gone. Status values are plain attribute types.
- ▶ The *id* attributes are gone, since they are persistence details.

▶ **Station–Bicycle:** This is now an aggregation, *Station* "0..1" o-- "0..*" *Bicycle*. [...]

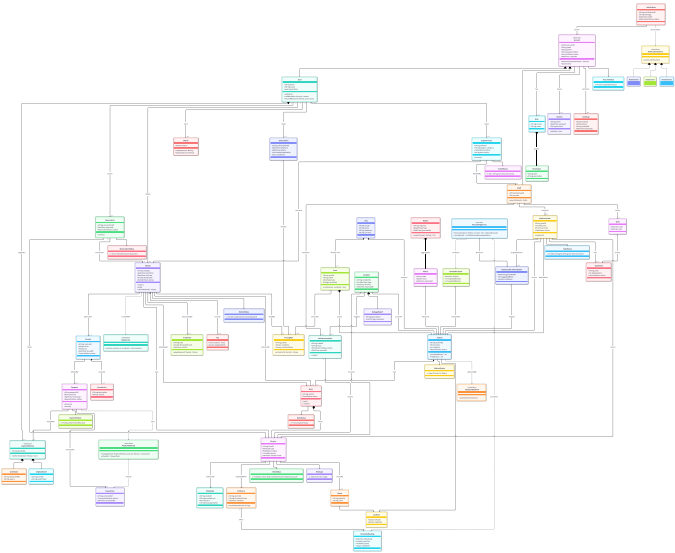
▶ **Object references:**

- ▶ *Member.plan* is replaced by the association *Member* --> *PricingPlan*.
- ▶ The old *Rebalancer.vehicle* attribute is removed.
- ▶ No attribute stores another object's *id*. Every link between classes is an association.

Reserve prompt (fresh session, after prompt #1)

“Now expand this into a complete enterprise architecture with all supporting subsystems — keep it a single Mermaid class diagram, no namespaces.”

AI's answer — the enterprise diagram



Zoom — lines from its Mermaid source

```
class Station {
  +String stationId
  +String name
  +int capacity
  +StationStatus status
  [...]
class Bicycle {
  +String bikeId
  +BikeType type
  +BikeStatus status
  +int batteryLevel
  [...]
class RebalanceTask {
  +String taskId
  +int bikeCount

City "1" *-- "1..*" Station : contains
Zone "1" o-- "*" Station : covers
Station "1" *-- "1..*" Dock : has
IoTDevice "1" --> "0..*" TelemetryReading : emits
Bicycle --> BikeStatus
Station --> StationStatus
RebalanceTask "*" --> "1..*" Bicycle : moves
```

Al's answer — what it said (reserve)

How the diagram is organized

- ▶ **Sections:** *With no namespaces, I used %% comment banners to group classes and relationships by subsystem. The subsystems are identity, geography, fleet, rentals, billing, payments, operations, support, notifications, and analytics/integration.*
- ▶ **Identity:** *User and Staff now share an abstract Account. Role-based access goes through Role and Permission. AuditLog covers accountability.*
- ▶ **Fleet and IoT:** *Each bike has an IoTDevice that emits TelemetryReadings. [...]*
- ▶ [...]
- ▶ **Enumerations:** *I included only the main enums to keep the diagram readable. Others (ZoneType, Severity, InvoiceStatus, and so on) are referenced as attribute types but not drawn.*

If the diagram is too dense to render well, I can split it into per-subsystem diagrams. [...]